

Guide de démarrage du TP en environnement Linux/Windows

Introduction

Ce guide s'adresse aux étudiants qui réalisent des Travaux Pratiques en programmation système en C, dans un environnement Linux natif ou un environnement émulé sous Windows via Cygwin.

Pourquoi Linux pour la programmation système ?

- Unix a été écrit en C, tout comme le noyau Linux.
- Le C permet d'interagir directement avec le système d'exploitation.
- Sous Linux, l'accès aux appels système est natif et conforme aux normes POSIX.

Environnement de travail recommandé

Pour garantir la cohérence des résultats et faciliter les manipulations, nous utiliserons :

- Linux (distribution Ubuntu, Debian, etc.)
- MacOS (qui repose aussi sur un noyau UNIX)
- Windows avec un émulateur comme Cygwin ou WSL

Utilisation de Cygwin sous Windows

Cygwin permet de simuler un environnement Linux sous Windows.

Site officiel : <https://www.cygwin.com>

Paquets recommandés lors de l'installation :

- gcc-core
- cygwin-devel
- libgcc1
- man-pages-posix (optionnel)

Structure des dossiers Linux / Cygwin

- /usr/bin : commandes système (ex : gcc, ps, man)
- /usr/include : fichiers d'en-tête système (unistd.h, stdio.h)
- /home/<utilisateur> : fichiers utilisateur
- /cygdrive/c : accès au disque C: depuis Cygwin

Commandes utiles

- **cd /cygdrive/c** : aller au disque C:
- **gcc fichier.c -o programme.o** : compiler un fichier C
- **./programme** : exécuter un programme compilé
- **ls -l** : lister les fichiers

Tester votre environnement

Vérifiez que gcc est bien installé avec la commande :

- **gcc --version**

Appels système fondamentaux :

Les quatre appels système suivants sont essentiels pour gérer les processus sous Linux :

- 1. fork()** : L'appel système `fork()` crée un nouveau processus en dupliquant le processus courant. Le processus fils reçoit une copie presque identique du parent, avec un PID différent. `fork()` retourne 0 au processus fils, et le PID du fils au processus parent.
- 2. exec()** : L'appel `exec()` permet à un processus de remplacer son image en mémoire par un nouveau programme. Il existe plusieurs variantes : `execl`, `execv`, `execle`, `execvp`, etc. Une fois `exec()` appelé avec succès, le processus courant est remplacé et ne continue plus son exécution initiale.
- 3. wait()** : `wait()` est utilisé par un processus parent pour attendre la fin d'un de ses processus fils. Il permet également de récupérer le code de sortie du fils et évite la création de processus zombies.
- 4. exit()** : L'appel système `exit()` termine le processus courant et renvoie un code d'état au processus parent. Ce code est ensuite récupérable via `wait()`. Toutes les ressources du processus sont libérées.

Travaux Pratiques :

Gestion des Processus sous Linux (Langage C)

Objectifs

L'objectif de ce TP est d'explorer les fondamentaux de la gestion des processus sous Linux en utilisant le langage C. Vous apprendrez à :

- Utiliser les primitives système pour manipuler les processus sous Linux.
- Créer des processus en utilisant `fork()`.
- Mettre en place des processus **zombies** et **orphelins**.
- Exécuter d'autres programmes à l'aide de `exec`.

1- Commandes de gestion des processus

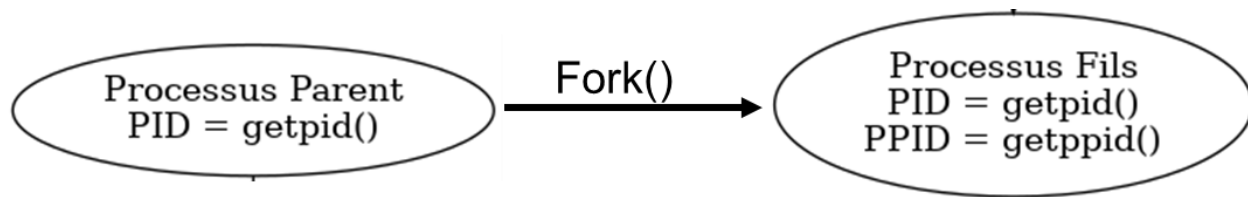
Sous Linux, plusieurs commandes permettent d'interagir avec les processus. Voici quelques-unes des plus utiles :

Commande	Description
<code>ps</code>	Affiche tous les processus en cours
<code>top</code> ou <code>htop</code>	Affiche les processus en temps réel
<code>kill -9 PID</code>	Tue un processus avec son PID
<code>jobs</code>	Affiche les processus en arrière-plan
<code>fg %n</code>	Ramène un processus en avant-plan
<code>bg %n</code>	Relance un processus en arrière-plan
<code>nice -n 10 ./programme</code>	Définit une priorité pour un processus

États de processus courants

Code	Nom complet	Description
R	Running	Le processus est en cours d'exécution ou prêt à être exécuté (dans la file d'attente du CPU).
S	Sleeping (Interruptible)	En veille, il attend un événement (comme une entrée clavier, une donnée réseau, etc.). Réveil possible.
D	Sleeping (Uninterruptible)	En veille non-interruptible (souvent pour des accès disque ou I/O). Ne peut pas être interrompu.
Z	Zombie	Le processus est terminé mais son père n'a pas encore lu son statut (via <code>wait()</code>).
T	Stopped	Le processus est arrêté (par exemple via <code>Ctrl + Z</code> ou <code>kill -STOP</code>).
X	Dead	(Rare) Processus terminé ou anormalement arrêté.
I	Idle (kernel thread)	Inactif, utilisé pour certains threads noyau (avec <code>ps -e</code> par ex.).

2- Création et manipulation des processus en C



Objectif

Comprendre comment un processus peut créer un autre processus sous Linux à l'aide de `fork()`, et comment chaque processus peut obtenir son identifiant (`PID`) et celui de son parent (`PPID`).

Contexte

Dans un système UNIX/Linux, **chaque programme exécuté correspond à un processus** identifié par un **PID** (Process ID).

Lorsqu'un processus crée un nouveau processus avec `fork()`, ce dernier devient son **fils**, et conserve un lien avec lui à travers son **PPID** (Parent Process ID).

Travail demandé

Crée un programme en langage C qui :

1. Utilise l'appel système `fork()` pour créer un **processus fils** ;
2. Affiche dans le **processus fils** :
 - son propre PID (avec `getpid()`),
 - le PID de son parent (avec `getppid()`).
3. Affiche dans le **processus parent** :
 - son propre PID,
 - le PID du fils (obtenu par `fork()`).

Aide technique

- `pid_t pid = fork();` retourne :
 - **0** dans le **fils**,
 - **n** le PID du **fils** dans le **père**,
 - **-1** en cas d'erreur.
- `getpid()` retourne le PID du processus courant.
- `getppid()` retourne le PID du parent.

Exemple de sortie attendue

```

Processus parent : PID=2314, fils=2315
Processus fils : PID=2315, PPID=2314
  
```

Correction

```
#include <stdio.h>          // Pour printf() et perror()
#include <unistd.h>          // Pour fork(), getpid(), getppid()

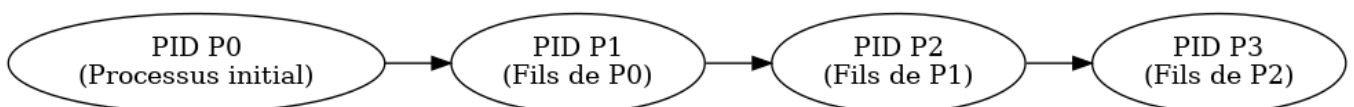
int main() {
    // Création d'un nouveau processus avec fork()
    // fork() retourne :
    // - 0 dans le processus fils
    // - le PID du fils dans le processus parent
    // - -1 en cas d'erreur
    pid_t pid = fork();

    // Gestion d'erreur si fork() échoue
    if (pid < 0) {
        perror("Erreur lors de la création du processus");
        return 1;
    }

    // Code exécuté par le processus fils
    if (pid == 0) {
        // getpid() : renvoie le PID du processus actuel
        // getppid() : renvoie le PID du parent
        printf("Processus fils : PID=%d, PPID=%d\n", getpid(), getppid());
    } else {
        // Code exécuté par le processus parent
        // pid contient ici le PID du fils
        printf("Processus parent : PID=%d, fils=%d\n", getpid(), pid);
    }

    return 0;
}
```

3- Création des processus selon une structure de filiation en chaîne



Objectif

Comprendre comment un processus peut **créer dynamiquement une chaîne de processus fils** successifs, chacun devenant parent du suivant. Visualiser la relation de **filiation séquentielle** à l'aide des PID et PPID.

Contexte

Grâce à `fork()`, un processus peut créer **un ou plusieurs processus fils**.

En ajoutant une **structure de boucle**, on peut faire en sorte que **chaque processus crée un nouveau processus à son tour**.

Cela construit une **chaîne de filiation** : chaque processus est à la fois **fils du précédent** et **père du suivant**.

Afin d'éviter les **processus zombies**, on utilise `wait()` dans chaque parent pour attendre la fin de son fils.

Travail demandé

Écris un programme C qui :

1. Crée **3 processus en chaîne** à l'aide d'une boucle `for`.
2. Dans chaque itération :
 - Le processus père crée un fils avec `fork()`
 - Le **fils continue la boucle** pour créer son propre fils
 - Le **père exécute `wait(NULL)`** et sort de la boucle
3. Affiche, dans chaque processus :
 - Son PID (`getpid()`)
 - Le PID de son parent (`getppid()`)
 - L'indice d'itération (`i`)

Concepts abordés

- `fork()` : création d'un nouveau processus
- `getpid()` / `getppid()` : identification du processus courant et de son parent
- `wait()` : synchronisation pour éviter les zombies
- Utilisation de `break` pour stopper la boucle dans le parent

Exemple de sortie attendue

```
Processus : PID=2345, Parent=2344 (Itération i=0)
Processus : PID=2346, Parent=2345 (Itération i=1)
Processus : PID=2347, Parent=2346 (Itération i=2)
```

Correction

```
#include <stdio.h>          // Pour printf
#include <unistd.h>         // Pour fork(), getpid(), getppid()
#include <sys/wait.h>       // Pour wait()

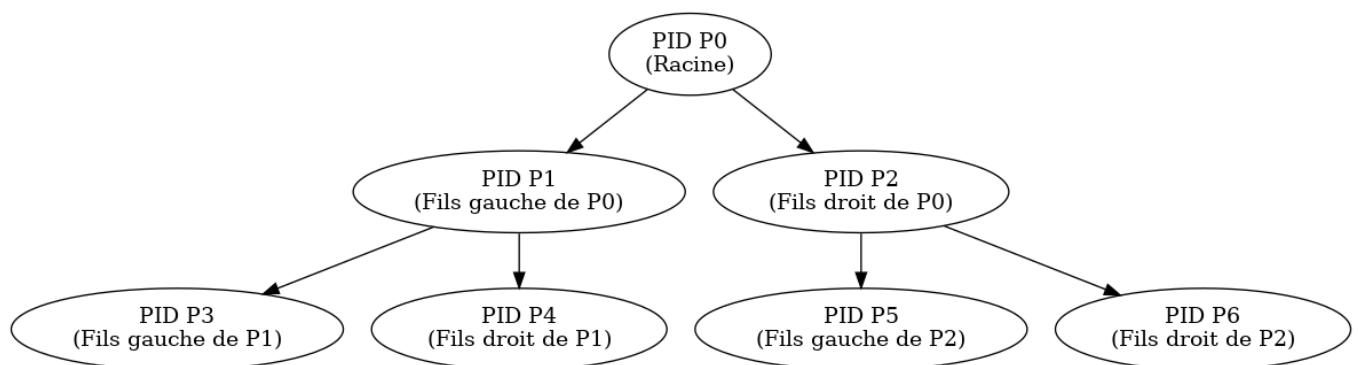
int main() {
    // Boucle pour créer 3 processus enfants successifs
    for (int i = 0; i < 3; i++) {
        pid_t pid = fork(); // Création d'un nouveau processus

        if (pid == 0) {
            // Ce code est exécuté uniquement par le processus fils
            printf("Processus : PID=%d, Parent=%d (Itération i=%d)\n",
getpid(), getppid(), i);

            // Le fils continue la boucle pour créer à son tour un enfant,
            // sauf si on veut l'arrêter
            // Ici on ne met pas de break car chaque fils peut créer son
            // propre sous-fils
        } else {
            // Ce code est exécuté uniquement par le processus parent
            wait(NULL); // Attend que le fils se termine pour éviter les
            zombies

            break; // Le parent se retire de la boucle après avoir créé un
            enfant
        }
    }
    return 0;
}
```

4- Création des processus selon une structure de filiation en arbre



Objectif

Comprendre comment créer une structure **en arbre binaire** de processus grâce à l'appel système `fork()` et à une fonction récursive. Visualiser l'arborescence des processus grâce aux PID et PPID.

Contexte

À la différence d'une filiation en **chaîne** où chaque processus engendre un seul fils, on peut construire une **arborescence binaire** où chaque processus a **deux fils** : un à gauche et un à droite.

Cette structure permet de modéliser la **création parallèle de processus**, très utilisée en architecture parallèle ou dans des algorithmes récursifs.

Travail demandé

Écris un programme C qui :

1. Affiche les informations du **processus racine** (`getpid()`).
2. Appelle une fonction `creer_arbre(niveau)` qui :
 - S'arrête si `niveau == 0`
 - Crée deux processus fils (gauche et droit)
 - Affiche le PID/PPID pour chacun
 - Appelle récursivement `creer_arbre(niveau-1)` dans chaque fils
 - Utilise `exit(0)` pour que les fils ne continuent pas l'exécution parentale
3. Utilise `waitpid()` pour éviter les processus zombies.

Concepts abordés

- Appels successifs à `fork()` pour créer une **structure en arbre**
- Utilisation de la **récursivité** pour créer des niveaux
- Distinction entre **fils gauche** et **fils droit**
- Gestion correcte des **terminaisons** avec `exit()` et synchronisation avec `waitpid()`

Exemple de sortie attendue (structure simplifiée niveau 2)

```
Processus racine : PID=2300
Fils gauche : PID=2301, PPID=2300
Fils droit : PID=2302, PPID=2300
Fils gauche : PID=2303, PPID=2301
Fils droit : PID=2304, PPID=2301
Fils gauche : PID=2305, PPID=2302
Fils droit : PID=2306, PPID=2302
```

Correction


```
#include <stdio.h>          // Pour printf()
#include <unistd.h>          // Pour fork(), getpid(), getppid()
#include <stdlib.h>          // Pour exit()
#include <sys/wait.h>        // Pour waitpid()

// Fonction récursive pour créer un arbre binaire de processus
void creer_arbre(int niveau) {
    // Condition d'arrêt : si le niveau est 0, on ne crée plus de processus
    if (niveau == 0) return;

    // Création du fils gauche
    pid_t pid1 = fork();

    if (pid1 == 0) {
        // Ce code est exécuté uniquement par le fils gauche
        printf("Fils gauche : PID=%d, PPID=%d\n", getpid(), getppid());

        // Appel récursif pour créer les enfants du fils gauche
        creer_arbre(niveau - 1);

        // Fin du processus fils pour éviter qu'il continue à forker à son
tour
        exit(0);
    }

    // Création du fils droit
    pid_t pid2 = fork();

    if (pid2 == 0) {
        // Ce code est exécuté uniquement par le fils droit
        printf("Fils droit : PID=%d, PPID=%d\n", getpid(), getppid());

        // Appel récursif pour créer les enfants du fils droit
        creer_arbre(niveau - 1);

        // Fin du processus fils
        exit(0);
    }

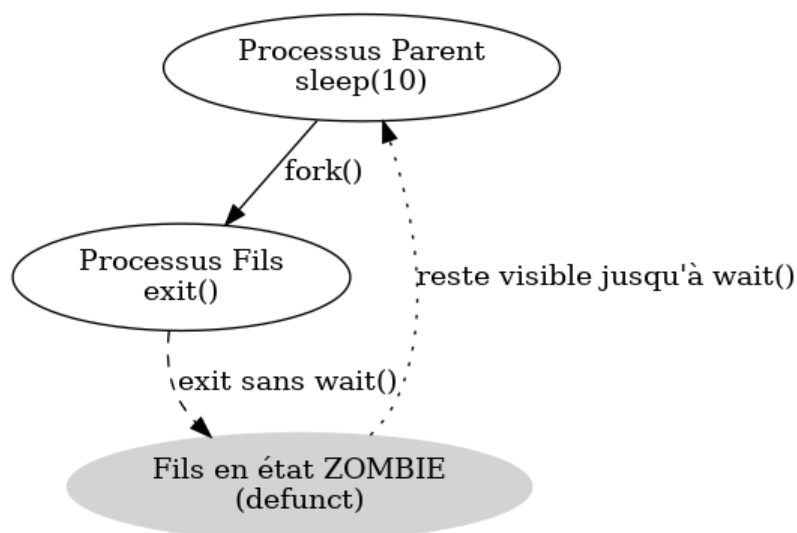
    // Le processus parent attend la fin des deux enfants pour éviter les
zombies
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);
}
```

```
int main() {
    // Affiche les informations du processus racine (le premier à
    s'exécuter)
    printf("Processus racine : PID=%d\n", getpid());

    // Création de l'arbre jusqu'au niveau 2
    creer_arbre(2);

    return 0;
}
```

5- Processus Zombie



Objectif

Créer un processus fils qui meurt avant son parent. Afin de comprendre ce qu'est un **processus zombie**, comment il se crée et comment l'observer avec des outils système (ps, top).

Contexte

Lorsqu'un **processus fils se termine**, il reste inscrit dans la table des processus jusqu'à ce que son **parent appelle wait()** pour lire son code de retour.

Tant que ce n'est pas fait, le processus est considéré comme **zombie** (état z ou defunct).

- Un zombie n'est **pas un bug**, mais un état temporaire tant que le père ne fait pas son travail de "récolte".
- **top** ou **ps -el** montre les zombies avec la lettre **Z**.

Travail demandé

Écris un programme en C qui :

1. Crée un **processus fils** avec `fork()`.
2. Le **fils se termine immédiatement** avec `exit(0)`, sans que le parent appelle `wait()`.
3. Le parent attend 10 secondes avec `sleep(10)`.
4. Le programme utilise `system("ps aux | grep 'defunct'")` pour afficher les processus zombies.

Concepts clés

- Un zombie **n'utilise pas de mémoire RAM**, mais reste dans la **table des processus**.
- Il disparaît uniquement si le **parent appelle `wait()`** ou si le **parent se termine** (et le zombie est alors récupéré par `init`).

Exemple de sortie attendue

```
Fils terminé : PID=2345
[sortie de la commande ps aux]
user      2345    ...  Z      ...  defunct
```

```
#include <stdio.h>          // Pour printf
#include <unistd.h>         // Pour fork(), sleep(), getpid()
#include <stdlib.h>         // Pour exit(), system()

int main() {
    // Création d'un processus fils
    pid_t pid = fork();

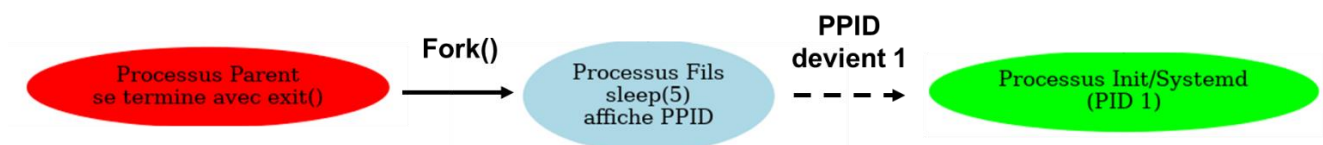
    // Code exécuté par le fils
    if (pid == 0) {
        // Le fils termine immédiatement
        printf("Fils terminé : PID=%d\n", getpid());
        exit(0); // Se termine sans que le père appelle wait()
    }

    // Code exécuté par le père
    // Il attend 10 secondes sans faire wait() pour le fils
    // Le fils devient un zombie pendant ce temps
    sleep(10);

    // Exécute une commande système pour afficher les processus zombies
    (defunct)
    system("ps aux | grep 'defunct'");

    return 0;
}
```

6- Processus Orphelin



Objectif

Comprendre le concept de **processus orphelin**, comment il est créé lorsqu'un processus parent meurt avant son fils, et comment il est récupéré par `init` ou `systemd`.

Contexte

Dans un système Linux, chaque processus a un **PPID** (Parent Process ID).

Lorsqu'un **père meurt avant son fils**, ce dernier devient un **orphelin**.

Il est alors automatiquement adopté par le **processus init (PID=1)** ou **systemd**, garantissant une bonne gestion des processus dans le système.

- Ce comportement montre **la robustesse du modèle de gestion de processus sous Linux**.
- Il est utile pour comprendre le rôle de `init` / `systemd` comme superviseur système.

Travail demandé

Écris un programme en C qui :

1. Crée un **processus fils** avec `fork()`.
2. Le **père se termine immédiatement** avec `exit(0)`.
3. Le **fils dort pendant 5 secondes** avec `sleep(5)` (le père aura disparu).
4. Le fils affiche :
 - son **PID**
 - son **PPID** (qui doit devenir 1 ou `systemd`)

Concepts clés

- L'orphelin **continue à vivre** même sans son père.
- Il est **automatiquement récupéré par `init` ou `systemd`**.
- Cela garantit qu'il sera bien terminé et qu'il ne devient pas zombie.

Exemple de sortie attendue

```
Je suis un orphelin. PID=2345, PPID=1
```

Correction

```
#include <stdio.h>          // Pour printf
#include <unistd.h>         // Pour fork(), sleep(), getpid(), getppid()
#include <stdlib.h>         // Pour exit()

int main() {
    // Création d'un processus fils
    pid_t pid = fork();

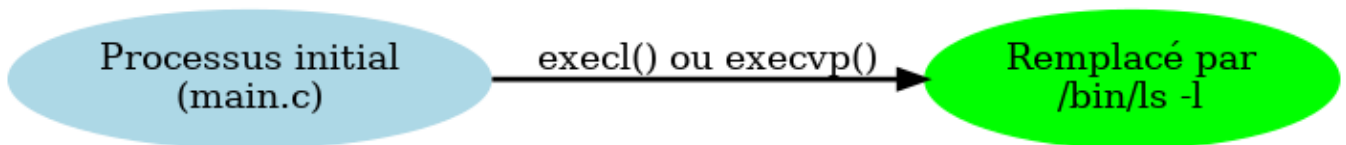
    // Code exécuté par le fils
    if (pid == 0) {
        // Le fils attend 5 secondes, le fils n'a plus de parent vivant.
        sleep(5);

        // Affiche son propre PID et celui de son parent (PPID)
        // Si le père est mort, le PPID sera 1 adopté par (init ou systemd)
        printf("Je suis un orphelin. PID=%d, PPID=%d\n", getpid(),
getppid());
    }
    // Code exécuté par le père
    else {
        // Le père se termine immédiatement sans attendre son fils
        // Ce qui rend le fils orphelin
        exit(0);
    }

    return 0;
}
```

7- Utilisation des variantes de processus

Objectif : Remplacer un processus par un autre.



Objectif

Comprendre le rôle de l'appel système `exec()` dans le remplacement complet du programme courant par un **nouveau programme**.

Contexte

L'appel système `exec()` permet à un processus d'abandonner totalement son code et d'exécuter un **autre programme à sa place, sans changer de PID**.

Cela est très utilisé pour **lancer un nouveau programme après un `fork()`** (par exemple dans un shell, un serveur, un pipeline...).

- `exec()` **ne crée pas de nouveau processus**, il remplace l'exécutable courant.
- Il est souvent utilisé **dans le fils après `fork()`** pour charger une autre commande.
- Il existe plusieurs variantes : `execl`, `execv`, `execle`, `execvp`...

Travail demandé

Écris un programme en C qui :

1. Utilise la fonction `execl()` pour **remplacer le programme courant** par la commande `ls -l`.
2. Affiche une erreur si `execl()` échoue (grâce à `perror()`).

Détails techniques

- `execl("/bin/ls", "ls", "-l", NULL) :`
 - `/bin/ls` est le chemin absolu vers la commande `ls`.
 - `"ls"` est le nom de la commande (argument `argv[0]`).
 - `"-l"` est un argument pour la commande.
 - `NULL` termine la liste des arguments.
- Si `execl()` réussit :
 - Le programme **est remplacé**,
 - Le code **après `execl()` ne s'exécute jamais**.
- Si `execl()` échoue :
 - On affiche un message avec `perror()`.

Exemple de sortie attendue

(total de fichiers affichés dans le répertoire courant, car ``ls -l`` a été lancé)

```
#include <stdio.h>          // Pour printf et perror
#include <unistd.h>         // Pour execl

int main() {
    // execl remplace l'image du processus courant par une nouvelle (ici /bin/ls)
    // Syntaxe : execl(path, arg0, arg1, ..., NULL)
    // arg0 est habituellement le nom de la commande elle-même ("ls")

    execl("/bin/ls", "ls", "-l", NULL);

    // Si execl réussit, ce code ne s'exécute jamais
    // En cas d'échec, on affiche une erreur
    perror("Erreur exec");

    return 1;
}
```